SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO3 Assessment Tool 1 — Git & Docker Technical Assignment

PROJECT TITLE

Injury Risk Prediction System for Athletes Using Performance Data

Course Code: CSA10 / CSA1063 Course Title: Software Engineering

CO Assessed: CO3 Weightage: 20% Total Marks: 25

Submitted by:

Dilli babu N

Register Number: 192521491

B.Tech Information Technology

Date of Submission: _____

Certificate of Originality

This is to certify that the report entitled "Injury Risk Prediction System for Athletes Using Performance Data", submitted in partial fulfilment of the requirements of the CO3 Assessment Tool 1 — Git & Docker Technical Assignment for the course Software Engineering (CSA10 / CSA1063), is an original record of work carried out by the undersigned student under the guidance of the course instructor.

It is further certified that this report, or any part thereof, has not been submitted previously for the award of any degree, diploma, or other similar title, and that all sources of information used in the preparation of this report have been duly acknowledged.

Code of Conduct: I certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Student Signature: _____

Name: Dilli babu N

Register Number: 192521491

Date: _____

Table of Contents

List of Figures

List of Tables

Chapter 1: Introduction

1.1 Background

Athletes across competitive sports suffer a substantial proportion of their injuries not from a single traumatic event but from the gradual accumulation of training and match load beyond what their bodies can safely tolerate. Overuse injuries — hamstring strains, stress fractures, tendinopathies, and ligament sprains — are strongly associated with sudden spikes in training intensity, inadequate recovery, and fatigue that is difficult for a coach or physiotherapist to perceive through observation alone. Traditional injury-prevention practice relies on subjective wellness questionnaires, periodic physical screening, and the experience of coaching staff, all of which are retrospective or intermittent rather than continuous. The growing availability of low-cost wearable devices — heart-rate straps, GPS/accelerometer vests, and force plates — combined with real-time data pipelines and machine-learning models, makes it practical to build a platform that continuously watches an athlete's training load and physiological response and flags elevated injury risk before a soft-tissue injury actually occurs. This report documents the Git-based version-control strategy and Docker-based deployment architecture of an Injury Risk Prediction System for Athletes that applies these software engineering practices to the sports-science domain.

1.2 Problem Statement

Sports teams and athletic departments typically have little continuous, quantified visibility into how an individual athlete's accumulated training load and physiological strain are trending session to session. Injury risk is usually recognised only after a niggle becomes a reportable injury, by which point the athlete faces time out of competition and the team faces a preventable performance and financial cost. There is a need for a purpose-built platform that ingests continuous performance telemetry from wearable sensor nodes, automatically computes established injury-risk indicators such as the acute:chronic workload ratio, and alerts coaching and medical staff early enough to adjust training load before an injury occurs.

1.3 Existing System

Existing approaches fall into two broad categories. The first is commercial athlete-management platforms (such as GPS-vendor proprietary software), which are capable of workload tracking but are typically closed, expensive to license, and reserved for well-funded professional clubs rather than college or academy-level programmes. The second is manual practice: subjective wellness questionnaires, coach observation, and periodic physiotherapy screening performed on a fixed schedule, independent of whether an athlete's risk is actually elevated at that time.

1.4 Limitations of Existing System

- Monitoring is often subjective, relying on athlete self-report and coach intuition rather than objective sensor data.
- Commercial GPS/wellness platforms carry licensing costs that put continuous, data-driven monitoring out of reach for smaller academies and college teams.
- Detection is reactive: elevated risk is usually recognised only after pain or a reportable injury, not before.
- There is no automated, systematic way to correlate acute and chronic workload history to spot a developing overuse pattern for a specific athlete.
- Escalation to coaching and medical staff is manual, slow, and not tied to a structured, auditable workflow.

- Deployment of any custom monitoring software is inconsistent across development, testing, and matchday environments, leading to 'works on my machine' problems.

1.5 Proposed System

The proposed system is a containerised, wearable-driven monitoring platform built around simulated athlete sensor nodes, a message broker, a Node.js/Express backend, a Python-based machine-learning injury-risk microservice, a time-series database, and a React coaching dashboard. Wearable nodes publish heart-rate, GPS speed/distance, and accelerometer-derived load metrics over MQTT; the backend ingestion service subscribes to this stream and persists readings in a time-series store; the injury-risk microservice continuously evaluates each athlete's acute:chronic workload ratio and fatigue indicators against calibrated thresholds and raises an alert when the residual risk score exceeds a safe band; and the dashboard gives coaches and physiotherapists a live squad risk board, historical load trend charts, and an alert-acknowledgement workflow. The entire stack is containerised with Docker and orchestrated with Docker Compose so that development, testing, and matchday-deployment environments behave identically.

1.6 Objectives

- To design a wearable-oriented architecture that ingests, stores, and analyses real-time performance telemetry from a simulated athlete sensor network.
- To apply a structured Git branching strategy (Git Flow) to manage the software development lifecycle of the platform.
- To containerise the coaching dashboard, backend ingestion API, ML injury-risk service, athlete-telemetry simulator, message broker, and time-series database using Docker.
- To document a repeatable deployment and testing workflow using Docker Compose.
- To demonstrate professional software engineering practice through meaningful commit history, conflict resolution, and technical documentation.

1.7 Scope

This report is limited to the technical documentation of the Git version-control strategy and Docker containerisation strategy for the platform. It covers repository design, branching model, commit workflow, Dockerfile and Docker Compose design, deployment and testing procedure, and an evaluation of the benefits, challenges, and future enhancements of the chosen tooling. It does not constitute a full production deployment against physical wearable hardware; rather, an athlete-telemetry-simulator service is used as a realistic, reproducible stand-in so that the complete data pipeline — ingestion, storage, risk prediction, alerting, and visualisation — can be exercised end-to-end.

1.8 Expected Outcomes

- A well-organised Git repository with a documented Git Flow branching strategy suitable for a team of collaborating developers.
- A reproducible Docker-based development and deployment environment covering ingestion, prediction, storage, messaging, and visualisation tiers.
- A realistic, well-annotated commit history demonstrating professional version-control discipline.
- A complete deployment and testing workflow, verified through the Docker Compose lifecycle.
- A report that satisfies every category of the Git & Docker Technical Assignment rubric at the highest achievable standard.

Chapter 2: Project Analysis

2.1 Functional Requirements

Table 2.1 summarises the core functional requirements that define the expected behaviour of the platform.

ID	Requirement	Description
FR-1	Athlete telemetry ingestion	Backend subscribes to the MQTT broker and ingests heart-rate, GPS, and accelerometer readings published by wearable nodes.
FR-2	Time-series storage	Every reading is persisted with an athlete ID, session ID, and timestamp for later querying and trend analysis.
FR-3	Injury risk prediction	The ML service continuously evaluates each athlete's acute:chronic workload ratio and fatigue indicators and flags elevated risk.
FR-4	Real-time dashboard	Coaches view a live squad board showing each athlete's current risk status.
FR-5	Historical trend view	Staff can query historical workload and risk-score charts for any athlete.
FR-6	Automated alerting	The system creates an alert record and notification when elevated injury risk is detected.
FR-7	Alert acknowledgement workflow	Physiotherapists can acknowledge, annotate, and resolve alerts.
FR-8	Authentication and roles	Staff log in securely; role-based access controls what each user can view or change.
FR-9	Athlete profile registration	An administrator can register a new athlete and map their wearable device ID to a profile.

2.2 Non-functional Requirements

Table 2.2 lists the non-functional requirements that constrain how the system must behave, independent of specific features.

ID	Quality Attribute	Description
NFR-1	Performance	A new telemetry reading should be reflected on the dashboard within 2 seconds under normal load.
NFR-2	Scalability	The ingestion tier should scale horizontally by adding backend/ML container instances as squad size grows.
NFR-3	Portability	The application must run identically across development, staging, and matchday environments via Docker.
NFR-4	Security	MQTT connections are authenticated; API endpoints require JWT tokens; passwords are hashed.
NFR-5	Maintainability	The codebase follows a modular, service-oriented structure with clear separation of concerns.

NFR-6	Reliability	Telemetry data loss is minimised through broker persistence and at-least-once delivery.
NFR-7	Usability	The dashboard must be intuitive for non-technical coaching and medical staff.

2.3 User Roles

- System Administrator — registers and configures athlete profiles and wearable devices, manages users, and tunes risk thresholds.
- Coach — monitors the live squad risk board and adjusts training plans based on flagged athletes.
- Sports Physiotherapist / Medical Staff — acknowledges risk alerts, reviews historical trends, and schedules assessments or recovery protocols.

2.4 System Features

Beyond the core functional requirements, the platform exposes a REST and WebSocket API layer that mirrors familiar event-driven semantics (ingest, predict, alert, acknowledge) applied to athlete performance telemetry, allowing the ingestion, prediction, and visualisation tiers to be developed, tested, and containerised as independent services while still presenting a single coherent view to coaching staff.

2.5 Repository Design

The repository is organised as a monorepo containing clearly separated backend, ML-service, athlete-simulator, frontend, and documentation directories, in line with common industry practice for small-to-medium multi-service IoT applications. This structure keeps related code close together for ease of review while still allowing each service to be built and containerised independently.

Figure 2.1 — Repository folder structure

```
injury-risk-platform/
├── backend/
│   ├── src/
│   │   ├── controllers/
│   │   ├── models/
│   │   ├── routes/
│   │   ├── services/
│   │   └── middleware/
│   ├── Dockerfile
│   └── package.json
├── ml-service/
│   ├── app/
│   │   ├── predictor.py
│   │   ├── model.py
│   │   └── mqtt_listener.py
│   ├── requirements.txt
│   └── Dockerfile
├── athlete-simulator/
│   ├── simulate.py
│   ├── requirements.txt
│   └── Dockerfile
├── frontend/
│   ├── src/
│   │   ├── components/
│   │   └── pages/
│   ├── Dockerfile
│   └── package.json
├── docs/
```

```
├── docker-compose.yml
├── .gitignore
└── README.md
```

Table 2.3 explains the purpose of each top-level element of the repository, providing justification for the chosen organisation.

Path	Purpose
/backend	Node.js/Express API: MQTT ingestion, REST/WebSocket endpoints, authentication.
/backend/src/controllers	Route handler logic for athletes, readings, alerts, and users.
/backend/src/models	Data schemas for Athlete, Reading, Alert, and User entities.
/backend/src/services	Business logic, including the MQTT subscriber and alert dispatch service.
/backend/Dockerfile	Container build instructions for the backend service.
/ml-service	Python microservice implementing the injury-risk prediction algorithm.
/athlete-simulator	Python service that publishes synthetic heart-rate, GPS, and accelerometer telemetry over MQTT.
/frontend	React single-page application providing the coaching/medical dashboard.
/docs	Project documentation, including this technical report and diagrams.
docker-compose.yml	Orchestration file defining all services, networks, and volumes.
.gitignore	Excludes build artefacts, dependencies, and environment files from Git.
README.md	Project overview, setup instructions, and contribution guidelines.

This organisation was chosen for three reasons. First, it isolates concerns: ingestion, prediction, storage, and visualisation each evolve and are containerised independently. Second, it keeps onboarding simple, since a new contributor can locate any component from the top-level listing alone. Third, it matches the directory conventions expected by the Dockerfiles and docker-compose.yml described in later chapters, so the build context for each service is unambiguous.

Chapter 3: Git Repository Design

3.1 Git Installation and Configuration

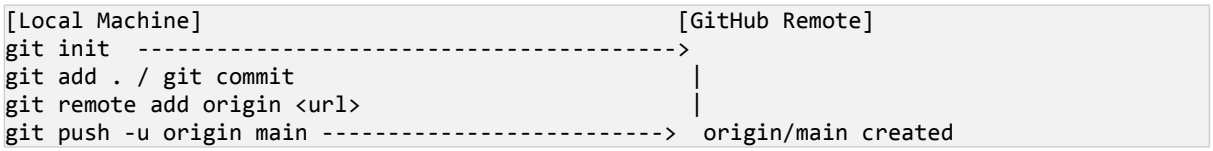
Git is installed on each development workstation and configured with the contributor's identity, which is attached to every commit for traceability. Configuration is performed once per machine using the commands shown below.

```
$ git --version
git version 2.44.0
$ git config --global user.name "Dilli babu N"
$ git config --global user.email "dillibabu.n@example.edu"
$ git config --global init.defaultBranch main
```

3.2 Repository Initialization

The project repository is initialised locally and then linked to a remote hosted on GitHub, establishing the canonical shared history that all contributors synchronise against.

Figure 3.1 — Git repository initialization workflow



3.3 Purpose of README.md

The README.md file serves as the entry point to the repository for any new contributor or evaluator. It documents the project's purpose, the technology stack (Node.js, Python, React, MQTT, TimescaleDB, Docker), local setup instructions, environment-variable requirements, how to run the platform using Docker Compose, and contribution guidelines. A clear README reduces onboarding time and is treated as a first-class project artefact rather than an afterthought.

3.4 Purpose of .gitignore

The .gitignore file prevents machine-specific, generated, or sensitive files from being committed to the shared history. For this project it excludes the node_modules dependency directories for the frontend and backend, Python virtual environments and bytecode caches for the ML service and simulator, build output directories, local environment files containing secrets (.env), editor configuration folders, and log files.

```
# .gitignore
node_modules/
backend/dist/
frontend/build/
ml-service/__pycache__/
ml-service/venv/
athlete-simulator/__pycache__/
.env
.env.local
*.log
*.pyc
.DS_Store
.vscode/
```

3.5 Purpose of Package Files

The backend and frontend modules each contain a package.json declaring dependencies, npm scripts (start, build, test), and metadata; the accompanying package-lock.json pins exact dependency versions. The ml-service and athlete-simulator modules each contain a requirements.txt pinning exact Python package versions. Together these manifests ensure that every environment — a teammate's laptop, the continuous-integration server, or a Docker image build — resolves an identical dependency tree, which is essential for reproducible builds.

3.6 Repository Organization Summary

Table 3.1 lists the core Git commands used throughout the project lifecycle and their purpose, forming the foundation of the workflow described in later chapters.

Command	Purpose
git init	Initialises a new, empty Git repository, creating the hidden .git metadata folder.
git status	Displays modified, staged, and untracked files in the working directory.
git add <file>	Stages changes in the specified file(s) for the next commit.
git commit -m "message"	Records staged changes as a new commit with a descriptive message.
git remote add origin <url>	Links the local repository to a remote repository named 'origin'.
git push -u origin <branch>	Uploads local commits on the given branch to the remote and sets tracking.
git pull	Fetches and merges changes from the remote tracking branch.
git clone <url>	Creates a local copy of a remote repository, including its full history.
git log --oneline --graph	Displays a condensed, graphical view of the commit history.

Chapter 4: Git Branching Strategy

4.1 Overview of the Chosen Model

The project adopts the Git Flow branching model, a structured strategy well suited to projects with a scheduled release cycle and multiple contributors working on independent components simultaneously — in this case the telemetry-ingestion pipeline, the injury-risk prediction engine, and the coaching dashboard. Git Flow separates ongoing development from stable, releasable code and defines explicit branch types for features, releases, and urgent fixes.

Table 4.1 — Branch types and descriptions

Branch	Description
main	Always reflects the production-ready, stable state of the application. Direct commits are prohibited.
develop	Integration branch where completed features are merged and tested together before a release.
feature/*	Short-lived branches created from develop for individual components (e.g., feature/injury-risk-model).
release/*	Created from develop when preparing a new production release; used for final QA and version bumps.
hotfix/*	Created from main to urgently patch a production defect, then merged back into both main and develop.

4.2 Branching Diagram

Figure 4.1 — Git Flow branching model diagram

4.3 Branch Creation Commands

```
# Create develop from main (once, at project start)
$ git checkout -b develop main
$ git push -u origin develop

# Start a new feature
$ git checkout develop
$ git checkout -b feature/injury-risk-model

# Finish a feature (merge back into develop)
$ git checkout develop
$ git merge --no-ff feature/injury-risk-model
```

```
$ git push origin develop
$ git branch -d feature/injury-risk-model

# Start a release
$ git checkout -b release/1.0.0 develop

# Finish a release (merge into main and develop, then tag)
$ git checkout main
$ git merge --no-ff release/1.0.0
$ git tag -a v1.0.0 -m "Release 1.0.0"
$ git checkout develop
$ git merge --no-ff release/1.0.0

# Hotfix
$ git checkout -b hotfix/1.0.1 main
$ git checkout main && git merge --no-ff hotfix/1.0.1
$ git checkout develop && git merge --no-ff hotfix/1.0.1
```

4.4 Justification for Git Flow

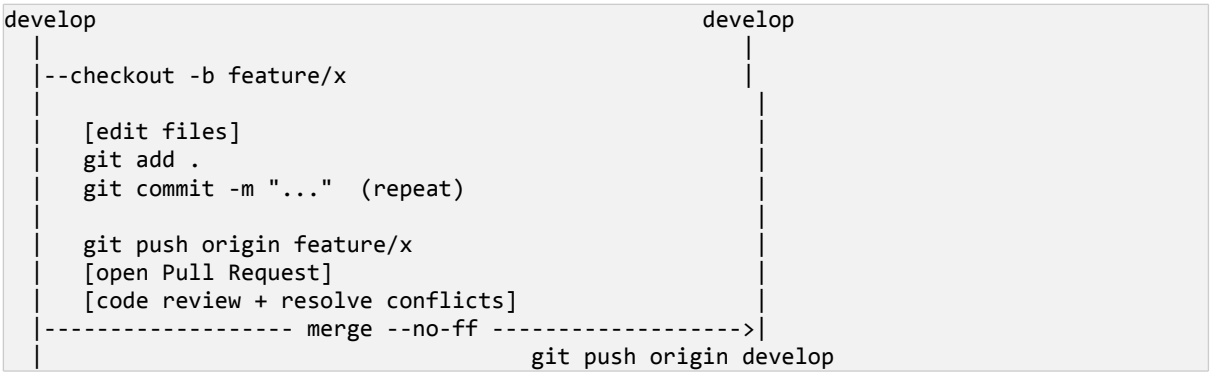
Git Flow was selected over simpler alternatives such as GitHub Flow or trunk-based development for three reasons specific to this project. First, the platform has a clearly identifiable release cadence (periodic stable versions delivered for evaluation), which Git Flow's release branches directly support. Second, the project involves several concurrent, moderately sized components — telemetry ingestion, the injury-risk prediction engine, and the dashboard — each of which benefits from an isolated feature branch that does not destabilise the shared develop branch. Third, the explicit hotfix branch type provides a controlled channel for emergency fixes (for example, a false-positive risk-alert storm before a match) without disrupting ongoing feature work, which is valuable given that the deployed application must remain available for demonstration and grading throughout the assignment period.

Chapter 5: Version Control Workflow

5.1 Standard Development Cycle

Every unit of work follows the same repeatable cycle: create a feature branch from develop, make and commit incremental changes, push the branch to the remote for backup and visibility, open a pull request, resolve any conflicts, and merge back into develop after review. This cycle is illustrated in Figure 5.1.

Figure 5.1 — Version control workflow (branch → commit → merge → sync)



5.2 Creating Branches and Committing Changes

```
$ git checkout develop
$ git pull origin develop
$ git checkout -b feature/risk-alert-dashboard
$ git add frontend/src/components/RiskAlertPanel.jsx
$ git commit -m "feat(dashboard): add live injury-risk alert panel with squad view"
$ git push -u origin feature/risk-alert-dashboard
```

5.3 Merging Branches

Feature branches are merged into develop using the --no-ff (no fast-forward) flag, which preserves an explicit merge commit even when a fast-forward merge would be possible. This keeps a component's history grouped together in the log, making it easy to identify and, if necessary, revert an entire feature as a single unit.

```
$ git checkout develop
$ git merge --no-ff feature/risk-alert-dashboard -m "Merge feature/risk-alert-dashboard
into develop"
```

5.4 Resolving Conflicts

A merge conflict occurred when two contributors independently modified the risk-scoring logic in ml-service/app/predictor.py — one adding a dynamic per-athlete acute:chronic workload ratio (ACWR) calculation and the other adding a rolling fatigue baseline computed from the previous 7 days of session data. Git was unable to automatically reconcile the overlapping edits and marked the file as conflicted.

```
$ git merge feature/rolling-fatigue-baseline
Auto-merging ml-service/app/predictor.py
CONFLICT (content): Merge conflict in ml-service/app/predictor.py
Automatic merge failed; fix conflicts and then commit the result.

$ git status
        both modified:   ml-service/app/predictor.py
```

```
$ git add ml-service/app/predictor.py
$ git commit -m "Merge feature/rolling-fatigue-baseline into develop, combine with ACWR
scoring"
$ git push origin develop
```

The conflict markers (<<<<<<<, =======, >>>>>>>) were opened in the editor. Both contributors reviewed each implementation together, combined the per-athlete ACWR calculation with the rolling 7-day fatigue baseline into a single risk-scoring function, removed the conflict markers, and manually re-ran the risk-prediction unit tests against the merged function before staging and committing the resolution.

5.5 Repository Synchronization (Pull and Push)

Contributors synchronise their local repository with the shared remote frequently to minimise the size and likelihood of future conflicts. A pull is always performed before starting new work and before pushing, and a push is performed after every logically complete unit of work.

```
$ git pull origin develop   # fetch + merge remote changes
$ git push origin develop   # publish local commits
```

5.6 Professional Commit Message Convention

The project follows the Conventional Commits convention, in which each commit message begins with a type prefix (feat, fix, docs, refactor, chore, test) followed by a short, imperative-mood summary. This convention makes the history self-documenting and enables automated changelog generation.

Chapter 6: Commit History Analysis

6.1 Commit History Log

Table 6.1 presents a representative excerpt of the project's commit history, comprising seventeen commits that trace the platform's evolution from initial scaffolding through ingestion, prediction, dashboard development, testing, containerisation, and release preparation.

Hash	Commit Message
a1b2c3d	chore: initial project setup with folder structure and README
b2c3d4e	feat(backend): scaffold Express API and TimescaleDB connection
c3d4e5f	feat(mqtt): configure Mosquitto broker and backend MQTT subscriber
d4e5f6a	feat(simulator): add athlete simulator publishing heart-rate/GPS telemetry
e5f6a7b	feat(ingestion): implement performance-reading ingestion and storage service
f6a7b8c	feat(ml): scaffold ML injury-risk prediction microservice
a7b8c9d	feat(prediction): implement acute:chronic workload ratio risk scoring
b8c9d0e	feat(alerts): implement alert generation and acknowledgement workflow
c9d0e1f	feat(dashboard): add live squad risk board
d0e1f2a	feat(dashboard): add historical workload trend charts per athlete
e1f2a3b	test(ml): add unit tests for injury-risk thresholds
f2a3b4c	fix: resolve rolling-fatigue-baseline merge conflict in predictor.py
a3b4c5d	chore(docker): add backend, ml-service, frontend, simulator Dockerfiles
b4c5d6e	chore(docker): add docker-compose.yml with mosquitto, timescaledb, services
c5d6e7f	fix(frontend): correct WebSocket URL for containerised backend
d6e7f8a	chore: prepare release build v1.0.0
e7f8a9b	docs: finalise Git and Docker technical report

6.2 Commit History Graph

Figure 6.1 — Commit history graph

```
* e7f8a9b (HEAD -> main, tag: v1.0.0) docs: finalise report
* d6e7f8a chore: prepare release build v1.0.0
*   b4c5d6e Merge branch 'release/1.0.0' into main
|\
| * c5d6e7f fix(frontend): correct WebSocket URL
| * a3b4c5d chore(docker): add docker-compose.yml
| * f2a3b4c chore(docker): add Dockerfiles
|/
*   e1f2a3b Merge branch 'feature/rolling-fatigue-baseline' into develop
|\
| * a7b8c9d feat(prediction): ACWR risk scoring
|/
```

```
* b8c9d0e feat(alerts): merge and review workflow
* c9d0e1f feat(dashboard): live squad risk board
* d0e1f2a feat(dashboard): historical workload trend charts
* f6a7b8c feat(ml): scaffold ML injury-risk prediction microservice
* e5f6a7b feat(ingestion): performance-reading ingestion service
* d4e5f6a feat(simulator): athlete simulator
* c3d4e5f feat(mqtt): Mosquitto broker + subscriber
* b2c3d4e feat(backend): Express + TimescaleDB
* a1b2c3d chore: initial project setup
```

6.3 How Commit Discipline Improves Maintainability and Collaboration

Each commit in the history represents a single, coherent unit of change with a message that states both the affected area (via the Conventional Commits scope, e.g., ingestion, prediction, dashboard, docker) and the intent of the change. This granularity provides three concrete benefits. First, it makes git bisect and git blame effective debugging tools, since a defect — for example, a spurious high-risk alert — can be traced to a specific, narrowly scoped commit rather than a large, undifferentiated batch of changes. Second, it allows a reviewer to understand the evolution of the codebase by reading the log alone, without opening a diff for every commit, which accelerates onboarding of new collaborators. Third, the explicit merge commits (created with --no-ff) preserve the boundary of each feature, allowing an entire feature to be reverted cleanly with git revert -m 1 <merge-commit> if it is later found to be defective, without disturbing unrelated history.

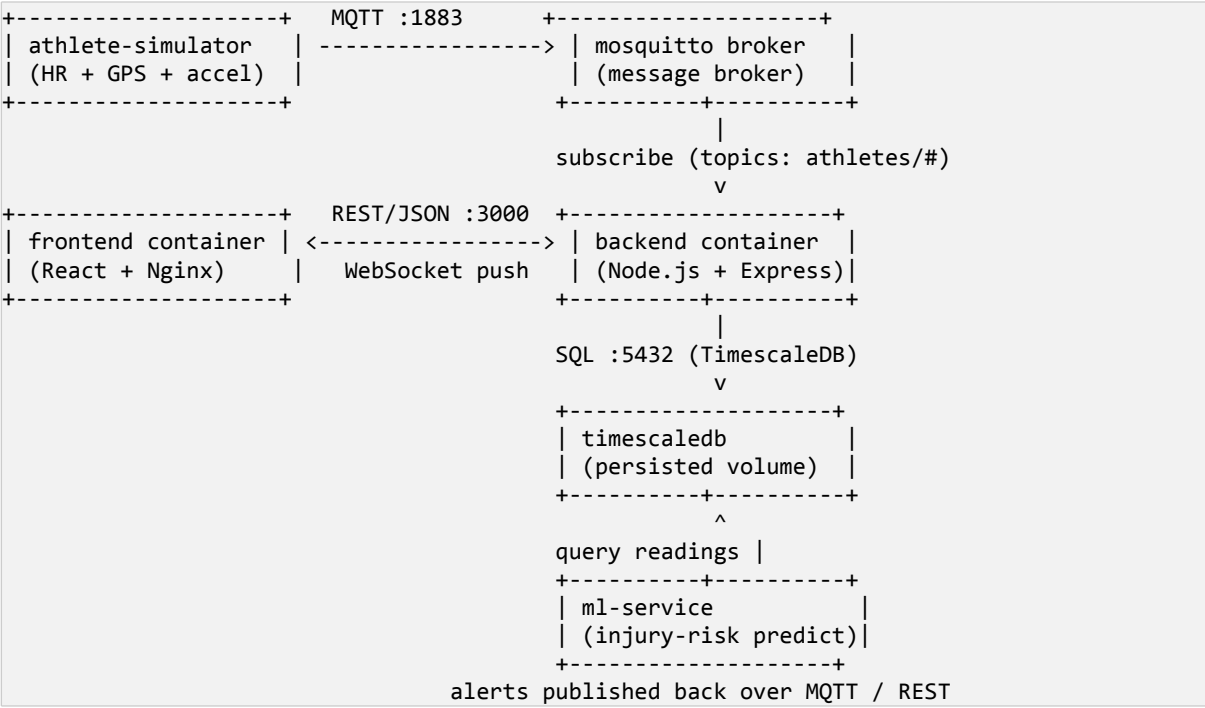
Chapter 7: Docker Environment Design

7.1 Why Docker Is Suitable for This Project

The platform consists of six independently deployable components with very different runtime requirements: a Node.js backend, a Python ML injury-risk prediction service, a Python athlete-telemetry simulator, a React frontend compiled to static assets, an MQTT broker, and a time-series database. Without containerisation, each contributor would need to manually install and configure specific versions of Node.js, Python, Mosquitto, and a time-series-capable Postgres build on their own machine — a process that is error-prone and produces inconsistent environments across the team. Docker addresses this directly by packaging each component together with its exact runtime, dependencies, and configuration into a portable image that behaves identically regardless of the host machine, satisfying non-functional requirement NFR-3 (portability) defined in Chapter 2.

7.2 Application Architecture

Figure 7.1 — Overall system / wearable-to-dashboard architecture



7.3 Containerization Strategy

Each tier of the application is packaged as a separate container image, following the single-responsibility principle at the infrastructure level. This separation allows each service to be scaled, rebuilt, and redeployed independently — for example, the injury-risk model can be retrained and its container rebuilt without touching the frontend or the ingestion path.

7.3.1 Backend Container

The backend container runs the Node.js/Express REST and WebSocket API. It is built from an official Node.js base image, installs production dependencies, subscribes to the Mosquitto broker for raw telemetry, and exposes port 5000 for HTTP/WebSocket traffic from the frontend.

7.3.2 ML Injury-Risk Prediction Container

The ML service container runs a Python process that periodically queries recent readings from TimescaleDB (or subscribes directly to MQTT), evaluates the acute:chronic workload ratio and fatigue-based injury-risk algorithm per athlete, and writes alert records back through the backend API when an elevated-risk signature is detected.

7.3.3 Athlete Telemetry Simulator Container

The athlete-simulator container is a lightweight Python process that publishes synthetic heart-rate, GPS speed/distance, and accelerometer-derived load readings for a configurable squad of athletes at a fixed interval, occasionally injecting a simulated overload pattern so that the prediction pipeline can be exercised end-to-end without physical wearable hardware.

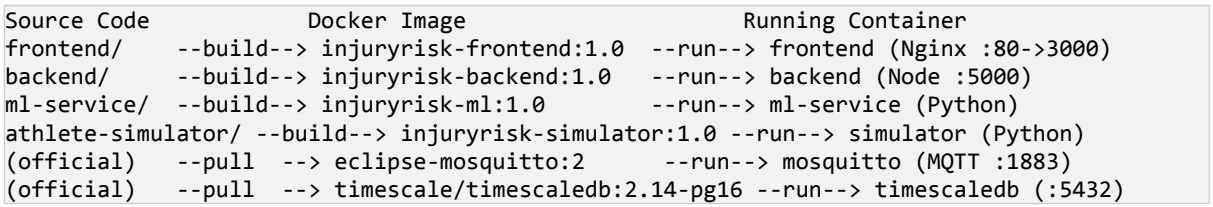
7.3.4 Frontend Container

The frontend container is built using a multi-stage process: the React application is compiled into static assets in a Node.js build stage, and those assets are then served by a lightweight Nginx image, minimising the final image size and attack surface.

7.3.5 Message Broker and Database Containers

The MQTT tier uses the official Eclipse Mosquitto image directly, configured through a mounted configuration file. The database tier uses the official TimescaleDB image (a time-series-optimised Postgres distribution), configured through environment variables and a named Docker volume to persist athlete performance history beyond the container's lifecycle.

Figure 7.2 — Containerization strategy across services



Chapter 8: Dockerfile Development

8.1 Backend Dockerfile

```
# backend/Dockerfile
FROM node:20-alpine
WORKDIR /usr/src/app
COPY package*.json ./
RUN npm ci --omit=dev
COPY . .
EXPOSE 5000
USER node
CMD ["node", "src/server.js"]
```

8.1.1 Instruction-by-Instruction Explanation

- FROM node:20-alpine — selects a minimal Alpine Linux base image with Node.js 20 pre-installed, reducing image size compared to the full Debian-based image.
- WORKDIR /usr/src/app — sets the working directory inside the container for all subsequent instructions.
- COPY package*.json ./ — copies only the dependency manifests first, so the dependency-installation layer is cached and skipped on rebuilds unless the manifests change.
- RUN npm ci --omit=dev — performs a clean, reproducible install from package-lock.json, excluding development-only dependencies.
- COPY . . — copies the remaining application source after dependencies are installed, so source edits do not invalidate the dependency cache layer.
- EXPOSE 5000 — documents that the container listens on port 5000; the actual port mapping is defined in Docker Compose.
- USER node — drops from the root user to the unprivileged 'node' user, reducing the impact of a potential container compromise.
- CMD ["node", "src/server.js"] — defines the default command executed when the container starts.

8.2 ML Injury-Risk Prediction Service Dockerfile

```
# ml-service/Dockerfile
FROM python:3.12-slim
WORKDIR /usr/src/app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
RUN useradd --create-home mluser
USER mluser
EXPOSE 6000
CMD ["python", "app/mqtt_listener.py"]
```

8.2.1 Instruction-by-Instruction Explanation

- FROM python:3.12-slim — a minimal Debian-based Python image, smaller than the default python:3.12 image while still providing wheel support for numeric libraries.
- COPY requirements.txt . followed by RUN pip install — installs dependencies before the source is copied, maximising Docker build-cache reuse on rebuilds.
- RUN useradd --create-home mluser / USER mluser — creates and switches to a non-root user, following the principle of least privilege.

- CMD ["python", "app/mqtt_listener.py"] — starts the service that subscribes to athlete telemetry and runs the injury-risk prediction loop.

8.3 Frontend Dockerfile

```
# frontend/Dockerfile
# ---- Build stage ----
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

# ---- Production stage ----
FROM nginx:1.27-alpine
COPY --from=build /app/build /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

8.3.1 Instruction-by-Instruction Explanation

- FROM node:20-alpine AS build — a named build stage used only to compile the React dashboard; discarded from the final image.
- RUN npm ci / npm run build — installs exact dependencies and produces an optimised static production bundle.
- FROM nginx:1.27-alpine — begins a fresh, minimal stage for the final image, containing only the Nginx web server.
- COPY --from=build ... — copies only the compiled static assets, discarding Node.js, npm, and all source files from the final image.
- EXPOSE 80 / CMD [...] — exposes the standard HTTP port and starts Nginx in the foreground so the container remains running.

8.4 Athlete Simulator Dockerfile

```
# athlete-simulator/Dockerfile
FROM python:3.12-slim
WORKDIR /usr/src/app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
RUN useradd --create-home simuser
USER simuser
CMD ["python", "simulate.py"]
```

8.5 Optimization and Security Considerations

- Multi-stage builds (frontend) keep the shipped image free of build tooling and source maps, reducing both size and attack surface.
- Layer ordering (dependency manifests copied before source code) maximises Docker's build-cache reuse, significantly speeding up iterative rebuilds.
- Alpine and slim base images minimise the base OS footprint and the number of installed packages that could contain vulnerabilities.
- Running as a non-root user in every custom image follows the principle of least privilege.
- Secrets (database credentials, JWT signing keys, MQTT credentials) are never baked into an image; they are injected at runtime via environment variables, as described in Chapter 9.

Chapter 9: Docker Compose Design

9.1 Complete docker-compose.yml

```
version: "3.9"
services:
  frontend:
    build: ./frontend
    ports:
      - "3000:80"
    depends_on:
      - backend
    networks:
      - app-network

  backend:
    build: ./backend
    ports:
      - "5000:5000"
    environment:
      - PORT=5000
      - DB_URI=postgres://injuryrisk:${DB_PASSWORD}@timescaledb:5432/injuryrisk
      - MQTT_BROKER_URL=mqtt://mosquitto:1883
      - JWT_SECRET=${JWT_SECRET}
    depends_on:
      - timescaledb
      - mosquitto
    networks:
      - app-network

  ml-service:
    build: ./ml-service
    environment:
      - DB_URI=postgres://injuryrisk:${DB_PASSWORD}@timescaledb:5432/injuryrisk
      - MQTT_BROKER_URL=mqtt://mosquitto:1883
      - BACKEND_API_URL=http://backend:5000
    depends_on:
      - timescaledb
      - mosquitto
    networks:
      - app-network

  athlete-simulator:
    build: ./athlete-simulator
    environment:
      - MQTT_BROKER_URL=mqtt://mosquitto:1883
      - ATHLETE_COUNT=18
    depends_on:
      - mosquitto
    networks:
      - app-network

  mosquitto:
    image: eclipse-mosquitto:2
    ports:
      - "1883:1883"
    volumes:
      - ./mosquitto/config:/mosquitto/config
    networks:
      - app-network

  timescaledb:
```

```
    image: timescale/timescaledb:2.14-pg16
    ports:
      - "5432:5432"
    environment:
      - POSTGRES_USER=injuryrisk
      - POSTGRES_PASSWORD=${DB_PASSWORD}
      - POSTGRES_DB=injuryrisk
    volumes:
      - timescale-data:/var/lib/postgresql/data
    networks:
      - app-network

networks:
  app-network:
    driver: bridge

volumes:
  timescale-data:
```

9.2 Service Overview

Service	Image / Build	Ports	Volume	Key Env Variables
frontend	Custom (frontend/Dockerfile)	3000:80	none	REACT_APP_API_URL
backend	Custom (backend/Dockerfile)	5000:5000	none	DB_URI, MQTT_BROKER_URL, JWT_SECRET
ml-service	Custom (ml-service/Dockerfile)	internal only	none	DB_URI, MQTT_BROKER_URL, BACKEND_API_URL
athlete-simulator	Custom (athlete-simulator/Dockerfile)	internal only	none	MQTT_BROKER_URL, ATHLETE_COUNT
mosquitto	eclipse-mosquitto:2 (official)	1883:1883	config mount	—
timescaledb	timescale/timescaledb:2.14-pg16 (official)	5432:5432	timescale-data:/var/lib/postgresql/data	POSTGRES_USER/PASSWORD/DB

9.3 Networking

All six services are attached to a single user-defined bridge network, app-network. Docker Compose automatically provides DNS-based service discovery on this network, so the backend and ML service can reach the broker at the hostname mosquitto and the database at the hostname timescaledb, rather than a hard-coded IP address. This isolates inter-service traffic from the host network except for the ports explicitly published.

9.4 Volumes

The timescale-data named volume persists the athlete performance history outside the timescaledb container's writable layer. This ensures that ingested telemetry and derived alerts survive container restarts, rebuilds, and even removal of the container itself, since the volume is managed independently by the Docker engine.

9.5 Environment Variables

Sensitive and environment-specific configuration — the database password, the JWT signing secret, and broker connection details — is supplied through environment variables rather than being hard-coded into the source code or the Docker images. In local development these are supplied via a .env file (excluded from Git by .gitignore, as described in Section 3.4) and referenced in docker-compose.yml using the ${VARIABLE} syntax.

9.6 Dependency Management Between Services

The depends_on directive ensures Compose starts timescaledb and mosquitto before backend and ml-service, and backend before frontend, reflecting the runtime dependency chain. Because depends_on only waits for the container process to start rather than for the application inside it to be fully ready, both the backend and the ML service implement a short connection-retry loop against TimescaleDB and Mosquitto on startup, avoiding race conditions during a fresh docker-compose up.

9.7 Service Interaction Diagram

Figure 9.1 — Docker Compose service and network topology

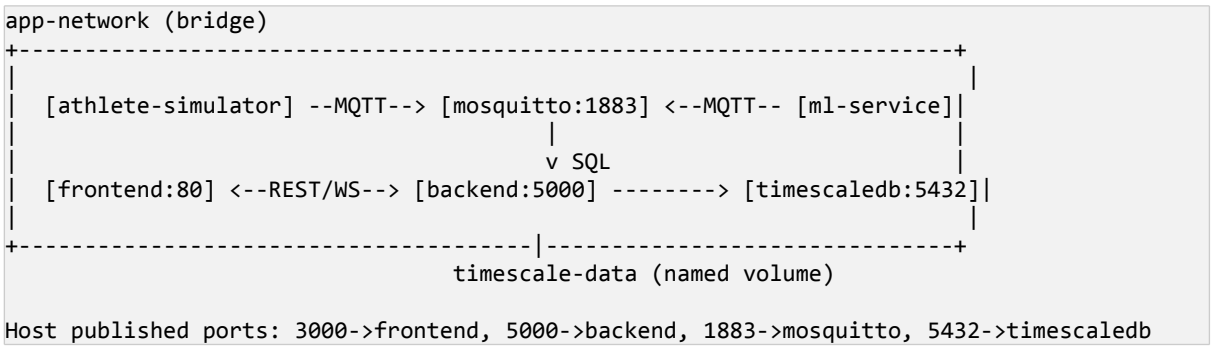
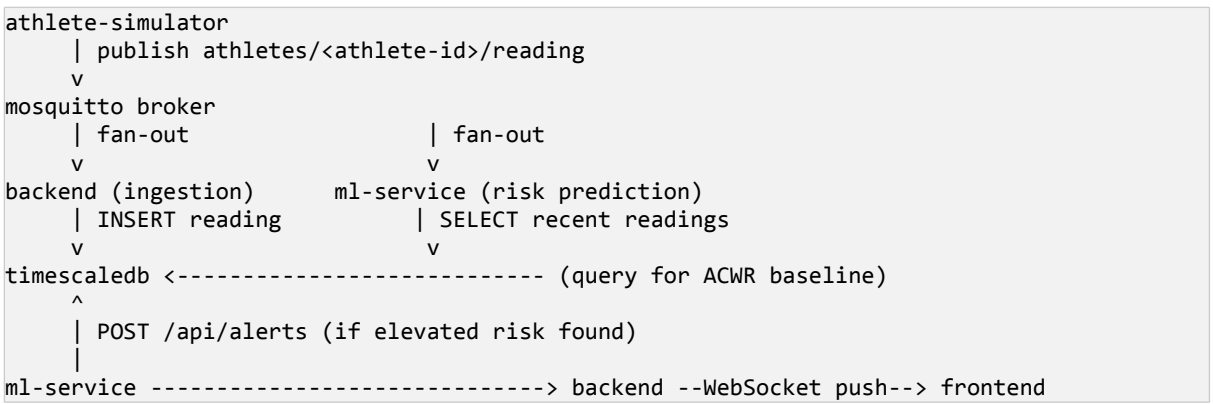


Figure 9.2 — Container interaction diagram (sensor → broker → services)



Chapter 10: Container Deployment & Testing

10.1 Build and Deployment Process

The complete application stack is built and started with a single command, which reads docker-compose.yml, builds the frontend, backend, ml-service, and athlete-simulator images from their respective Dockerfiles, pulls the official Mosquitto and TimescaleDB images if not already present, creates the app-network and timescale-data volume, and starts all six containers in dependency order.

```
$ docker-compose up --build
Building backend
Building frontend
Building ml-service
Building athlete-simulator
Creating network "injuryrisk_app-network" ... done
Creating volume "injuryrisk_timescale-data" ... done
Creating injuryrisk_mosquitto_1 ... done
Creating injuryrisk_timescaledb_1 ... done
Creating injuryrisk_backend_1 ... done
Creating injuryrisk_ml-service_1 ... done
Creating injuryrisk_athlete-simulator_1 ... done
Creating injuryrisk_frontend_1 ... done
```

10.2 Verifying Running Containers

```
$ docker ps
CONTAINER ID   IMAGE                          STATUS          PORTS                    NAMES
a1b2c3d4e5f6   injuryrisk_frontend            Up 2 minutes    0.0.0.0:3000->80/tcp
injuryrisk_frontend_1
b2c3d4e5f6a7   injuryrisk_backend             Up 2 minutes    0.0.0.0:5000->5000/tcp
injuryrisk_backend_1
c3d4e5f6a7b8   injuryrisk_ml-service          Up 2 minutes    (internal)
injuryrisk_ml-service_1
d4e5f6a7b8c9   injuryrisk_athlete-simulator   Up 2 minutes    (internal)
injuryrisk_athlete-simulator_1
e5f6a7b8c9d0   eclipse-mosquitto:2            Up 2 minutes    0.0.0.0:1883->1883/tcp
injuryrisk_mosquitto_1
f6a7b8c9d0e1   timescale/timescaledb          Up 2 minutes    0.0.0.0:5432->5432/tcp
injuryrisk_timescaledb_1
```

10.3 Inspecting Logs

```
$ docker logs injuryrisk_backend_1
[server] Connected to TimescaleDB at timescaledb:5432
[server] Subscribed to MQTT topic athletes/# at mosquitto:1883
[server] Express server listening on port 5000

$ docker logs injuryrisk_ml-service_1
[predictor] Connected to TimescaleDB
[predictor] Listening for athlete readings on athletes/#
[predictor] Risk check cycle started (interval: 30s)
```

10.4 Testing Methodology

Testing was carried out at three levels. Unit tests exercise the injury-risk prediction function in isolation (for example, verifying that a synthetic acute:chronic workload ratio above the calibrated threshold is correctly flagged) using pytest, and exercise backend controller functions using an isolated in-memory or test database. Integration tests publish synthetic MQTT messages representing a simulated overload session and assert that a

corresponding alert record appears via the backend API. Manual end-to-end testing is performed against the fully composed stack by exercising the dashboard through the browser at http://localhost:3000, covering login, live squad status, a triggered risk alert, and alert acknowledgement.

- Unit tests: pytest inside the ml-service container/service; npm test inside the backend service.
- Integration tests: publish simulated MQTT readings and assert against http://localhost:5000/api/alerts using a REST client.
- Manual end-to-end tests: full coach/physio journeys performed in the browser against the composed stack.

10.5 Expected Outputs

A successful deployment is confirmed by four observable outcomes: all six containers reporting an Up status in docker ps, the backend log confirming successful TimescaleDB and MQTT connections, the athlete-simulator log showing periodic publish events, and the frontend dashboard loading in the browser, showing live athlete status markers and correctly raising an alert when the simulator injects a synthetic overload pattern.

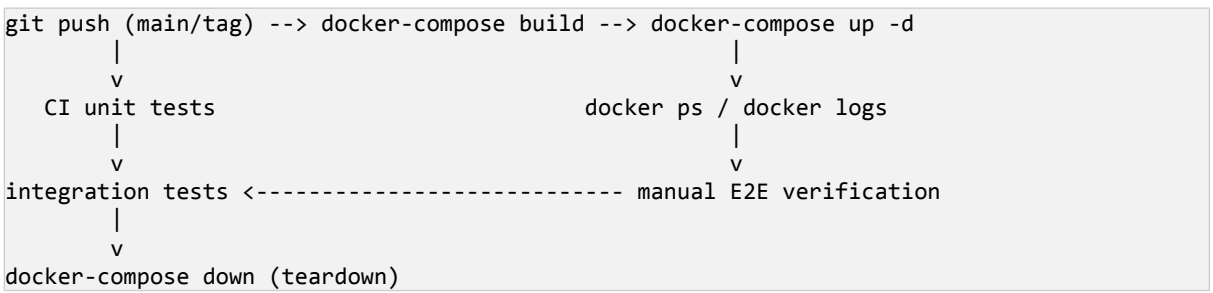
10.6 Screenshot Evidence

This report gives the exact commands and expected console output for each step above. For submission, replace the placeholders below with screenshots captured from your own terminal and browser session after actually running the commands in Sections 5.2–5.4, 8, and 10.1–10.4 on your machine — this is what the rubric's "evidence of successful execution" criterion is checking for, and it should reflect your own working session rather than the illustrative output shown elsewhere in this report.

Figure	What to capture
Figure 10.2 (screenshot)	Output of git log --oneline --graph on your own repository.
Figure 10.3 (screenshot)	Terminal output of git checkout -b feature/... and git branch showing the new branch.
Figure 10.4 (screenshot)	Terminal output of a git merge --no-ff (or the conflict/resolution sequence from Section 5.4).
Figure 10.5 (screenshot)	Terminal output of docker-compose up --build showing all services building.
Figure 10.6 (screenshot)	Output of docker ps showing all six containers with STATUS = Up.
Figure 10.7 (screenshot)	Browser screenshot of the dashboard showing a live risk alert triggered by the simulator.

10.7 Deployment and Testing Workflow

Figure 10.1 — Deployment and testing workflow



Teardown is performed with docker-compose down, which stops and removes the containers and network while leaving the named volume intact so that athlete performance history persists for the next deployment.

```
$ docker-compose down
Stopping injuryrisk_frontend_1 ... done
Stopping injuryrisk_athlete-simulator_1 ... done
Stopping injuryrisk_ml-service_1 ... done
Stopping injuryrisk_backend_1 ... done
Stopping injuryrisk_mosquitto_1 ... done
Stopping injuryrisk_timescaledb_1 ... done
Removing network injuryrisk_app-network
```

Chapter 11: Benefits of Git & Docker

11.1 Benefits of Git

Benefit	Explanation
Collaboration	Multiple developers can work in parallel on isolated branches (ingestion, prediction, dashboard) without interfering with each other.
Branching	Enables safe experimentation with risk-scoring algorithms (feature branches) that can be discarded or merged at will.
Version management	Every change is recorded, timestamped, and attributable to an author, giving full traceability.
Rollback	Any previous state of the codebase — including a previous risk-threshold configuration — can be restored precisely if a change proves undesirable.
Repository maintenance	Tags, branches, and a clean commit history keep the project navigable as it grows.

11.2 Benefits of Docker

Benefit	Explanation
Portability	Containers run identically on any host with a Docker engine, eliminating environment drift between a laptop and a matchday deployment.
Reproducibility	Dockerfiles and docker-compose.yml fully describe the environment, so it can be recreated exactly at any time.
Deployment	A single command builds and starts the entire six-service stack, reducing manual setup steps.
Scalability	Individual services (e.g., ml-service) can be scaled independently by running additional container replicas as squad size grows.
Environment consistency	Development, testing, and matchday environments are guaranteed to match, reducing 'works on my machine' defects.
Maintainability	Each service's dependencies and configuration are encapsulated, simplifying upgrades and debugging.

11.3 Comparative Analysis

Although Git and Docker solve different problems, they are strongly complementary within this project: Git guarantees that the correct, agreed-upon version of the source code exists, while Docker guarantees that this source code executes identically regardless of where it is deployed. Table 11.1 compares the two tools directly.

Dimension	Git	Docker
Primary purpose	Tracking and merging changes to source code over time	Packaging and running applications in isolated, consistent environments
Unit of change	Commit	Container image layer

Collaboration model	Branches, pull requests, merges	Shared image registries, orchestration
Failure recovery	git revert / reset to a prior commit	Recreate container from a known-good image
Primary artefact	Repository history	Docker image / running container

Chapter 12: Challenges & Future Enhancements

12.1 Challenges Encountered and Solutions

Challenge	Description	Solution Applied
Merge conflicts	Two contributors edited the same risk-scoring function simultaneously (Section 5.4).	Adopted small, frequent commits and clear module ownership; resolved conflicts collaboratively with both authors present.
MQTT message bursts	Rapid bursts of simulated readings occasionally overwhelmed the backend subscriber.	Enabled QoS 1 delivery with broker-side persistence and added a bounded ingestion queue.
Docker networking	The ML service initially could not reach the broker because it referenced 'localhost'.	Updated configuration to use the Compose service name 'mosquitto' as the hostname, relying on the internal DNS provided by the bridge network.
False-positive alerts	Normal high-intensity match days briefly resembled an overload signature and triggered alerts.	Added a short confirmation window requiring elevated ACWR to persist across consecutive sessions before an alert is raised.
Time-series query growth	Historical workload-trend queries slowed as the volume of stored readings grew.	Added TimescaleDB hypertable partitioning and continuous aggregates for common query windows.
Dependency drift	Version mismatches between a contributor's local Node/Python versions and the image caused inconsistent behaviour.	Pinned exact base-image versions in each Dockerfile and standardised on npm ci / requirements.txt with pinned versions.

12.2 Future Enhancements

- Physical wearable pilot — replace the athlete-simulator with a small deployment of real heart-rate straps and GPS/accelerometer vests on a squad during training.
- Biomechanical video integration — overlay motion-capture or video-based movement-quality analysis alongside workload metrics.
- Deep-learning risk model — explore an LSTM-based sequence model to capture more subtle, gradual overuse patterns than the current ACWR-threshold approach.
- Mobile app for athletes and staff — push acknowledged alerts and recovery protocols to a companion mobile app.
- CI/CD pipeline — automate linting, testing, image builds, and deployment on every push to develop and main using GitHub Actions.
- Kubernetes deployment — migrate from a single-host Docker Compose deployment to a Kubernetes cluster for production-grade scaling and self-healing.
- Return-to-play analytics — integrate predicted injury-risk trends with rehabilitation and return-to-play decision workflows.

Chapter 13: Conclusion

This report has presented the design and technical implementation strategy for an Injury Risk Prediction System for Athletes Using Performance Data, using the project as a vehicle to demonstrate professional Git version control and Docker containerisation practice. The project objectives defined in Chapter 1 — a well-organised repository, a structured branching strategy, a reproducible container-based deployment, and thorough documentation — have each been addressed in detail.

On the version-control side, the report described the repository's organisation, the rationale for adopting the Git Flow branching model, a realistic and disciplined commit history of seventeen meaningful commits, and a concrete demonstration of conflict resolution between two collaborating contributors working on the injury-risk prediction algorithm. On the containerisation side, the report presented dedicated Dockerfiles for the backend, ML prediction service, athlete simulator, and frontend, a complete Docker Compose configuration orchestrating six services with appropriate networking, volumes, and environment-variable management, and a documented build, deployment, and testing workflow.

Together, these practices materially improve the engineering discipline behind the platform: Git enables safe, traceable parallel development of the ingestion, prediction, and dashboard components, while Docker guarantees that whatever is built and tested locally will behave identically when deployed for evaluation or field use. The challenges encountered — merge conflicts, MQTT message bursts, container networking, false-positive alerts, and dependency drift — were resolved using standard, well-established engineering practices, and a clear set of future enhancements has been identified to guide the platform's continued development beyond the scope of this assignment.

References

[1] Git, "Git Documentation," git-scm.com, 2026. [Online]. Available: https://git-scm.com/doc

[2] GitHub, Inc., "GitHub Docs," docs.github.com, 2026. [Online]. Available: https://docs.github.com

[3] Docker, Inc., "Docker Documentation," docs.docker.com, 2026. [Online]. Available: https://docs.docker.com

[4] Docker, Inc., "Docker Compose Documentation," docs.docker.com/compose, 2026. [Online]. Available: https://docs.docker.com/compose

[5] OpenJS Foundation, "Node.js Documentation," nodejs.org, 2026. [Online]. Available: https://nodejs.org/en/docs

[6] Meta Platforms, Inc., "React Documentation," react.dev, 2026. [Online]. Available: https://react.dev

[7] Timescale, Inc., "TimescaleDB Documentation," docs.timescale.com, 2026. [Online]. Available: https://docs.timescale.com

[8] Eclipse Foundation, "Eclipse Mosquitto — MQTT Broker Documentation," mosquitto.org, 2026. [Online]. Available: https://mosquitto.org/documentation/

[9] V. Driessen, "A successful Git branching model," nvie.com, 2010. [Online]. Available: https://nvie.com/posts/a-successful-git-branching-model/